Lecture 4

How to write code?

Should I be searching online for code?

Of course 👌

You will do that anyway

Try to find good advice 😇

It is like learning "chords" for the ukulele.

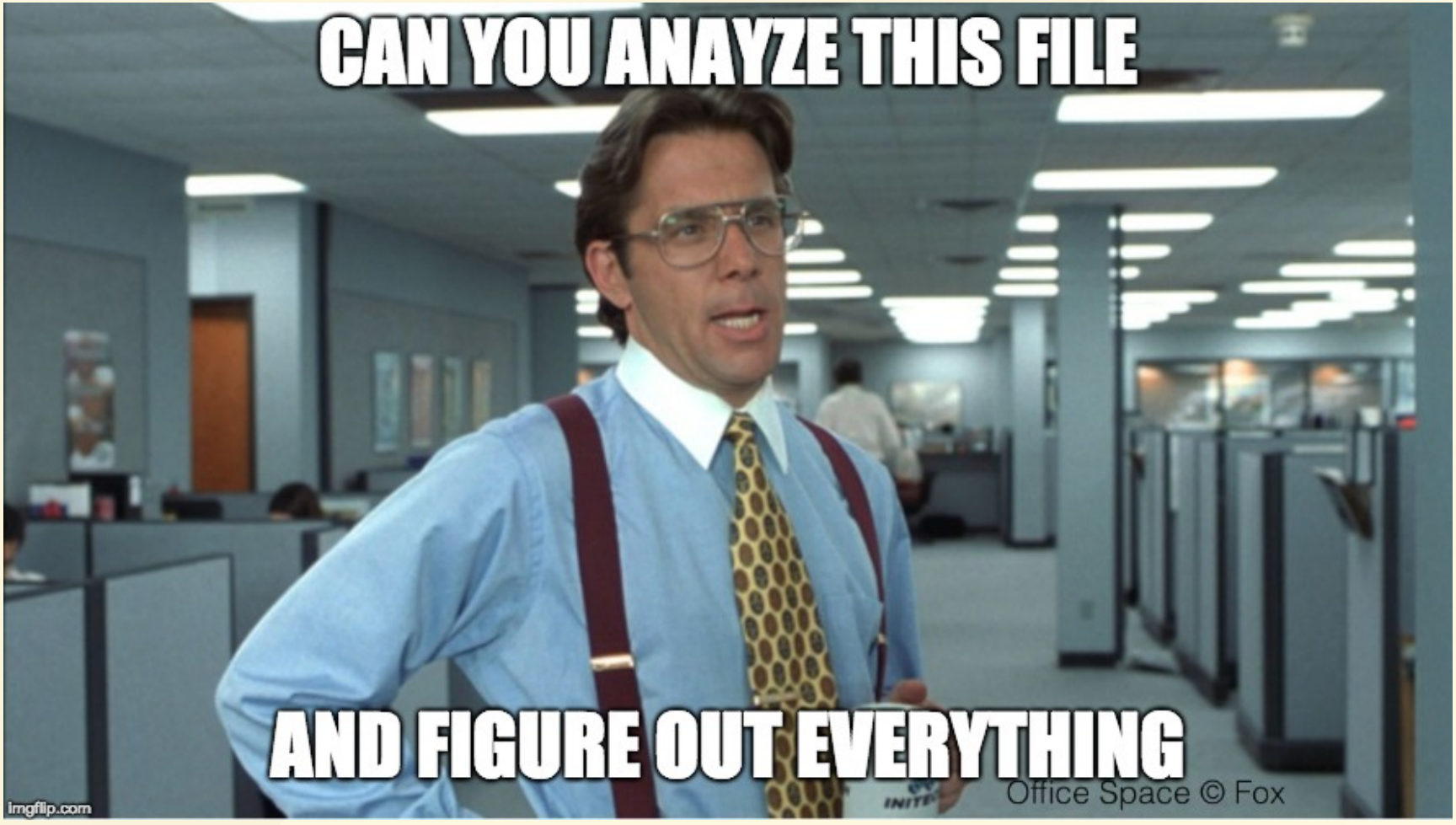
Find chords, analyze chords, adopt chords

The best place to find code examples for a specific task.

http://wwww.stackoverflow.com

Don't worry Google will get you there.

Data Science in a nutshell



Characterize the problem 👁

The most important feature

---

1. Can you *fully model* all the data first then *solve* it?

---

2. Is it a *stream* oriented problem (*endless* data) where we have to *continuosly* solve it?

Solutions styles

1. Fully modeled solutions are often seem simpler to explain and demonstrate. Loads up all data into memory.

2. Streaming solutions are more abstract but usually end up as much more robust to adapt and will be simpler to reuse.

Best solutions: stream as much as possible, then build a full model from a subset.

How do I read a column from a file into python?

What are the conceptual units 🍎

You'd be surprised just how many problems are built of the following:

1. Read a file
2. Put values into data structure
3. Manipulate the data structure

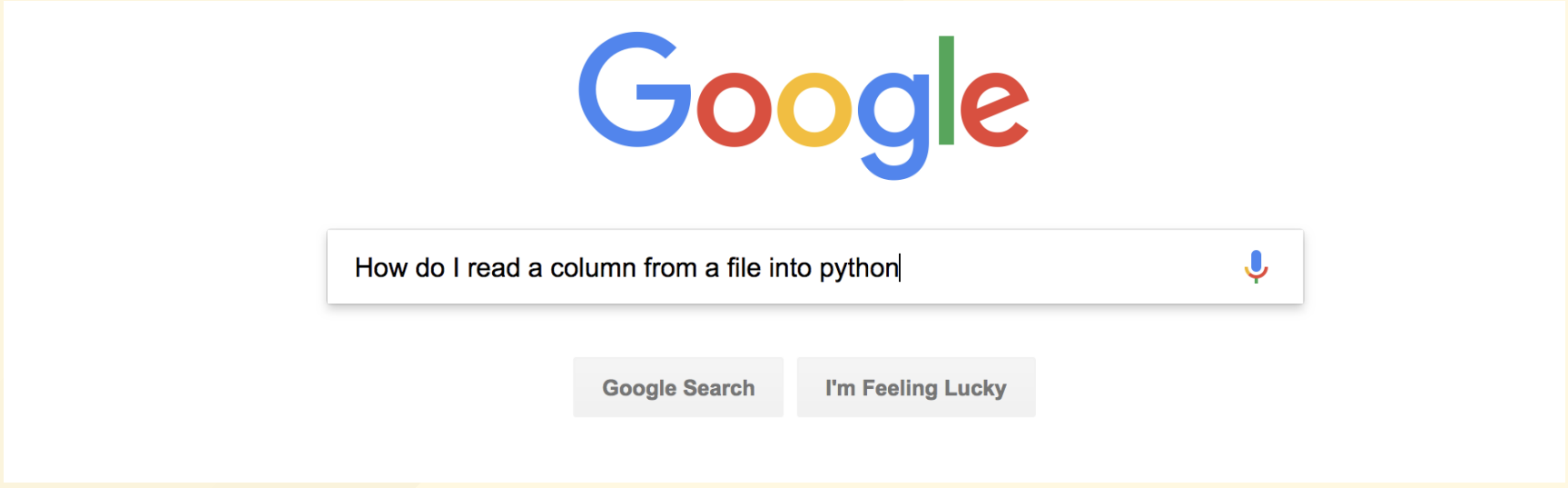
If you have built the right data structure, you can get the answer quickly.

So how do I read a column from a file into python?

How do others do it? 👸 🤴 🎅 👼

Google It You Will

Everybody googles for code all the time.



Here is our first hit

4 Answers

active oldest votes

10

```
f=open(file,"r")
lines=f.readlines()
result=[]
for x in lines:
    result.append(x.split(' ')[1])
f.close()
```

You can do the same using a list comprehension

```
print [x.split(' ')[1] for x in open(file).readlines()]
```

Docs on `split()`

> `string.split(s[, sep[, maxsplit]])`
>
> Return a list of the words of the string `s` . If the optional second argument sep is absent or None, the words are separated by arbitrary strings of whitespace characters (space, tab, newline, return, formfeed). If the second argument sep is present and not None, it specifies a string to be used as the word separator. The returned list will then have one more item than the number of non-overlapping occurrences of the separator in the string.

So, you can omit the space I used and do just `x.split()` but this will also remove tabs and newlines, be aware of that.

share improve this answer

edited May 13 '15 at 13:47

answered May 13 '15 at 13:39



ForceBru

16.6k ● 4 ● 31 ● 51

This is what the answer states

```
f=open(file,"r")
lines=f.readlines()
result=[]
for x in lines:
    result.append(x.split(' ')[1])
f.close()
```

Good code comes with input examples:

```
5 10 6
6 20 1
7 30 4
...
```

Losing your TABS 😈

When you copy/paste from the internet tabs are often turned into spaces ....

```
A	B
```

What is between `A` and `B` four spaces or a single tab?

Sigh ... We put a man on the moon but we can't copy a tabs ...

Your editor must have the "**view whitespace**" option. Check what did you copy over.

⚠️ Code examples may be written for Python 2

Some code may work with no changes.

But some code will fail.

- `print` is a keyword in Python 2
- `print()` is a function in Python 3
- The names for some modules have changed.

When code fails spectacularly - it might be a Python 2 example.

The code example uses some file

Put the following into `data.txt`

```
5 10 6
6 20 1
7 30 4
8 40 3
9 23 1
4 13 6
```

Place it in the same folder as your code.

Is this good code?

Kinda. Sorta.

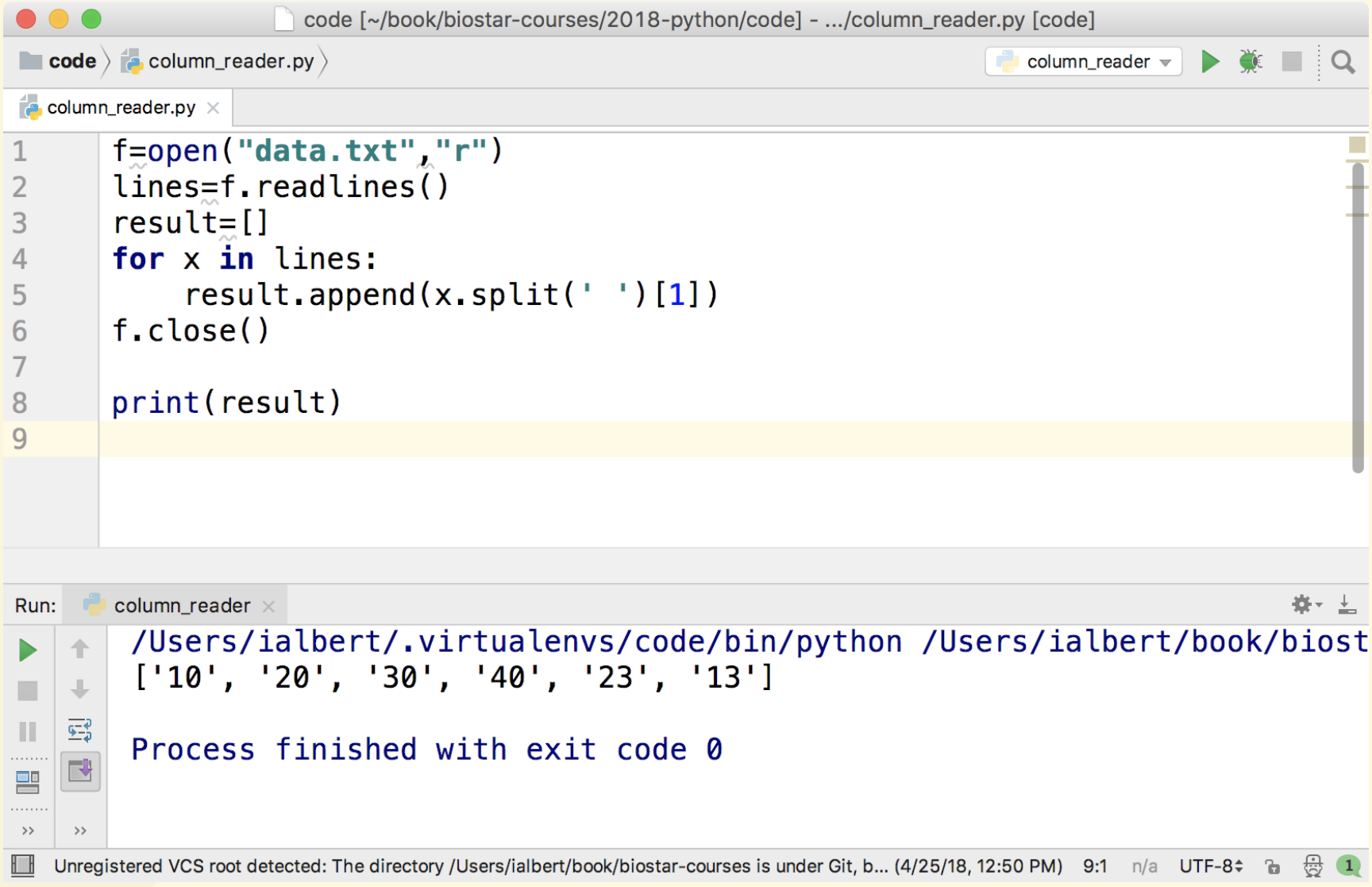
```
f=open("data.txt","r")
lines=f.readlines()
result=[]
for x in lines:
    result.append(x.split(' ')[1])
f.close()
```

It works. And it is a good start.

First rule: **Make it work!**

Clean up later.

It's alive


code [~/book/biostar-courses/2018-python/code] - .../column_reader.py [code]
code
column_reader.py
column_reader
column_reader.py
1 f=open("data.txt","r")
2 lines=f.readlines()
3 result=[]
4 for x in lines:
5 result.append(x.split(' ')[1])
6 f.close()
7
8 print(result)
9
Run:
column_reader
/Users/ialbert/.virtualenvs/code/bin/python /Users/ialbert/book/biost
['10', '20', '30', '40', '23', '13']
Process finished with exit code 0
Unregistered VCS root detected: The directory /Users/ialbert/book/biostar-courses is under Git, b... (4/25/18, 12:50 PM) 9:1 n/a UTF-8

Learn to program in stages

1. Make it work somehow
2. Maybe you copied random code from random places
3. Work on it until you get it to *work*. Phew!

... **then later** ...

4. Analyze and understand what takes place *exactly*
5. Perhaps there is a simpler way to do it
6. Keep modifying it one step at a time
7. You will make this code your own!

The anatomy of a block

Indentation (white space at the beginning of a line) has special meaning in Python.

A block designates a "unit" that belongs together

```
code 1

block:
    code 2
    code 3

code 4
```

`code 2` and `code 3` will run together in the "block"

Blocks are indicated via indentation

😈 What's with the "indentation" hate?

Lots of people *hate* python because of this feature.

Why?

They feel it interferes with their thought process.

It does interefere, it makes thinking easier!

It was designed to do that!

Python has had usability studies that show code is more readable this way.

```
values = [1, 2, 3]

for x in values:
    y = x + 1
    z = x + 2
    print(x, y, z)
```

Delineates which code gets executed "together"

So why the hate?

It is because you HAVE to DO IT!

Nothing gets the *rebels* going more than a mandate.

```
values = [1, 2, 3]

for x in values:
    y = x + 1
    z = x + 2
    print(x, y, z)
```

There will be people that get worked up about indentation

```
¯\_(ツ)_/¯
```

How do other languages do it

The Python loop over an array:

```
for x in values:
    print(x)
```

May look like this in `C`:

```
maxval = sizeof(values)/sizeof(values[0])
for( i = 0; i < maxval; i = i + 1 ){
  printf(&values[i]);
}
```

Always use SPACES for indentation!

Never use TABS!

Why does Pyton even allow TABS? 😈

Back to our program 

```
for x in lines:
    result.append(x.split(' ')[1])
```

Rewrite to do fewer things at once

```
for x in lines:
    y = x.split(' ')[1]
    result.append(y)
```

Rewrite to make it readable

```
for line in lines:
    column = x.split(' ')
    select = column[1]
    result.append(select)
```

Continuously rework it 🚲

Here is where the quick turnaround helps. Keep re-running the program.

Comment regions out. Print out what each "thing" is. Find out what it does.

Break it down to components you do understand.

```
result = []
for line in stream:
    print (line)
    #words = line.split(' ')
    #print(words)
```

Handy "forever" pattern ❤️

This pattern is **very** common. Learn it use it.

1. Create emtpy list
2. Fill up your list

```
# Create an empty list.
result = []

for line in stream:
    # Grow your list
    # Append  to the end in each loop.
    result.append(1)
```

What does `result` contain?

Looping construct with `for`

Find the sum of the column:

```
# Create an empty list.
result = []

for line in stream:
    column = line.split(' ')
    select = int(column[1])
    result.append(select)

print(result)
print(max(result), min(result), sum(result))
```

Same problem, many solutions 🌼

There are always many solutions each with its own tradeoffs.

Complexity, performance, efficiency, simplicity, beauty, elegance.

But first of all: **MAKE IT WORK!**

Then fix it up.

What would be my best version? 👌

What should be an optimal "column" reader look like? What is a version that I think is the best?

Here is what I think. Don't worry about not getting it yet though. This is what the course is about.

```
import csv

def extract(words):
    return int(words[1])

stream = open("data.txt")
reader = csv.reader(stream, delimiter=" ")
result = map(extract, reader)
```

Are there other solutions

Of course.

Everbody needs to read columns



Pandas: Python Data Science Framework 🐼

Loading data with Pandas

```
import pandas

# Read file into a panda data frame.
df = pandas.read_csv("data.txt", delimiter=' ')

# Take all rows of the second column.
values = df.iloc[:, 1]

# Turn the resulting type into a list.
values = list(values)

# Print the list.
print(values)
```

Note that here the entire file is loaded into memory!